



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO de COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



DOCUMENTO: MANUAL TÉCNICO

Alumno: 320109766

GRUPOS DE TEORÍA: 05

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 24/11/2025

Geometría

- La fachada y su jerarquía.

Fachada



Importar modelos

```
///// Modelos proy final /////  
  
Model puerta;  
/// Cuarto 1 ///  
Model casa;
```

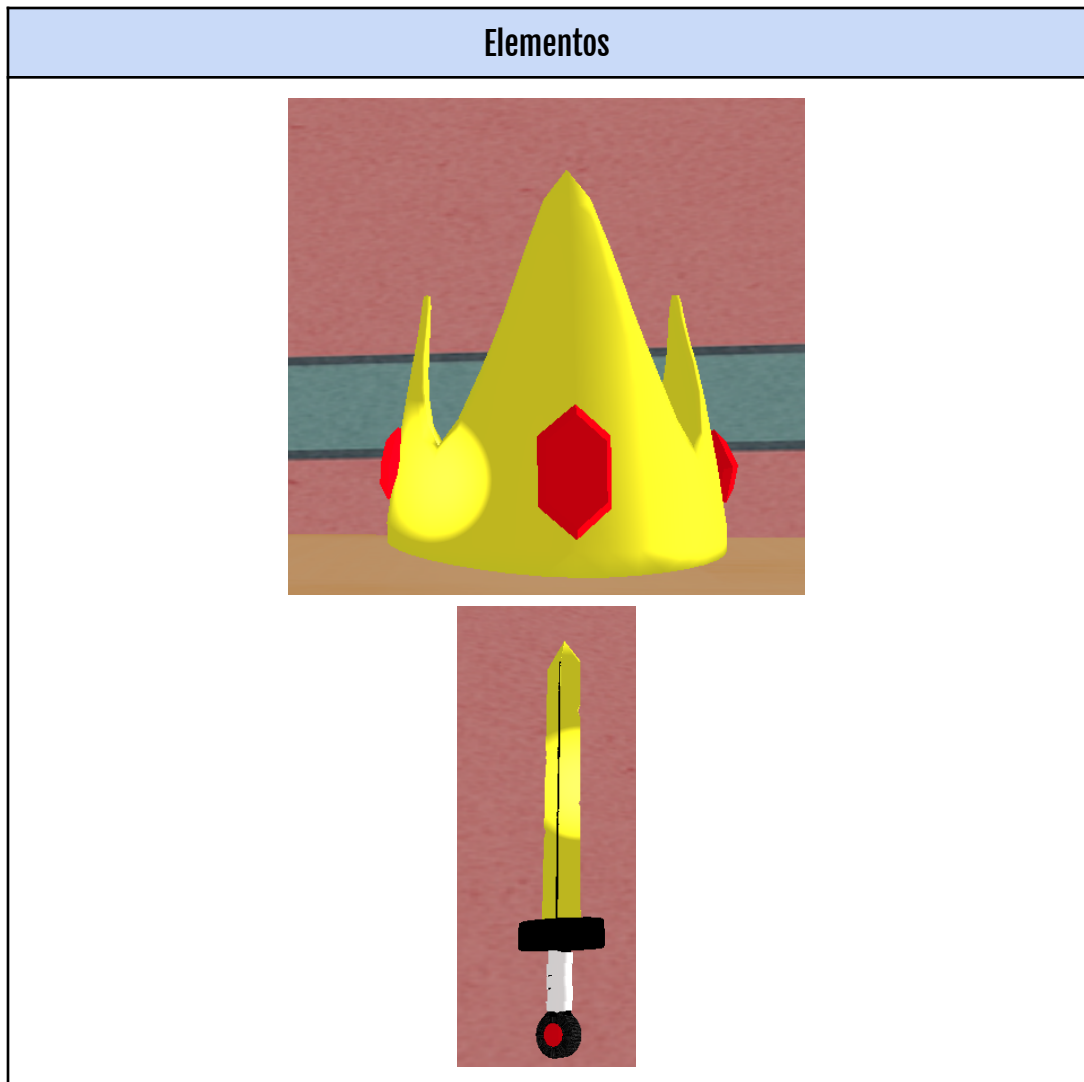
```
// Proy final  
puerta = Model();  
puerta.LoadModel("Models/puertafinal.obj");  
  
//cuarto 1  
casa = Model();  
casa.LoadModel("Models/casa.obj");
```

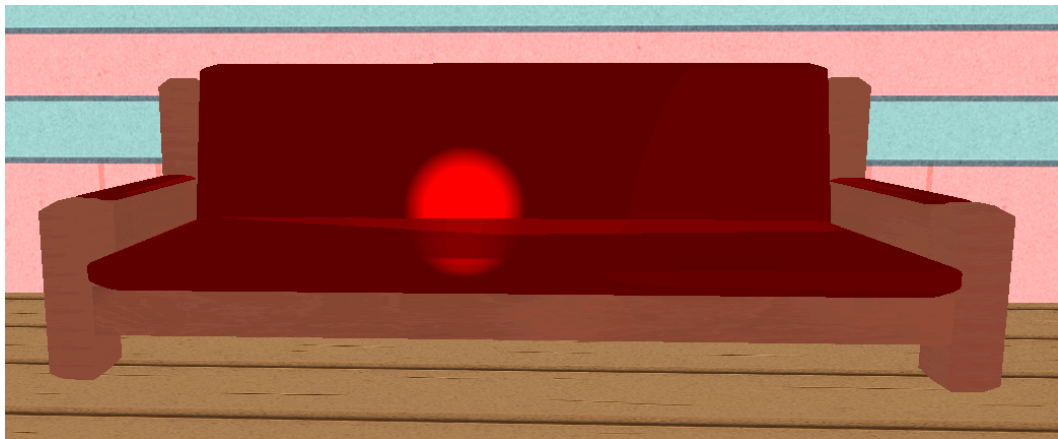
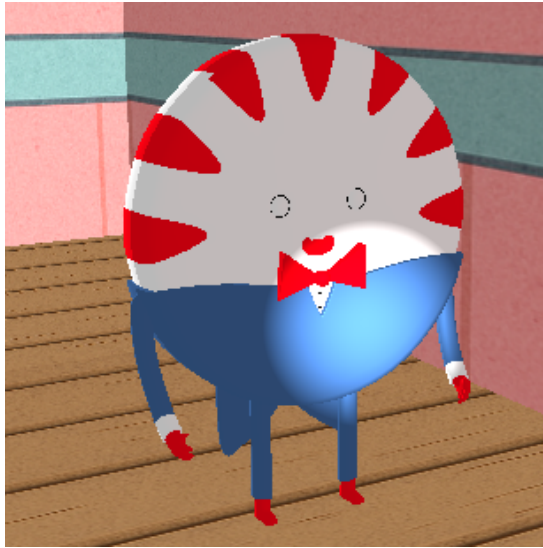
Implementación

```
//Casa  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(15.5f, 5.5f, 1.5f));  
modelaux = model;  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
casa.RenderModel();
```

```
//Puerta 1
model = modelaux;
model = glm::translate(model, glm::vec3(-1.7f, -3.1f, -20.0f));
model = glm::rotate(model, rotpuerta * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta.RenderModel();
```

- Elementos del cuarto 1





Importar modelos

```
Model sillon2;  
Model espada;  
Model mentita;  
Model mano;  
Model mesa;  
Model corona;
```

```
sillon2 = Model();  
sillon2.LoadModel("Models/sillon2.obj");  
  
espada = Model();  
espada.LoadModel("Models/espada.obj");  
  
mentita = Model();  
mentita.LoadModel("Models/mentita.obj");  
  
mano = Model();  
mano.LoadModel("Models/manomentita.obj");  
  
mesa = Model();  
mesa.LoadModel("Models/mesa.obj");
```

Implementación

```

//Espada
model = modelaux;
model = glm::translate(model, glm::vec3(6.8f, 0.5f, 5.5f));
model = glm::translate(model, glm::vec3(0.0f, 0.0 + 0.5 * sin(glm::radians(2 * angulovariable)), 6.0));
model = glm::rotate(model, anguloEspada * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
espada.RenderModel();

//Corona
model = modelaux;
model = glm::translate(model, glm::vec3(3.8f, -2.0f, 5.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
corona.RenderModel();

//Mesa
model = modelaux;
model = glm::translate(model, glm::vec3(3.5f, -2.65f, 5.5f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
mesa.RenderModel();

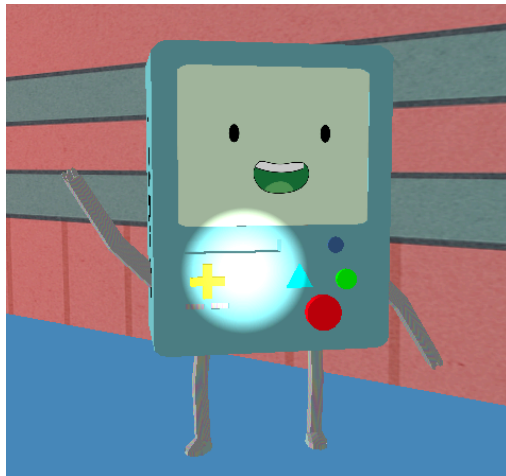
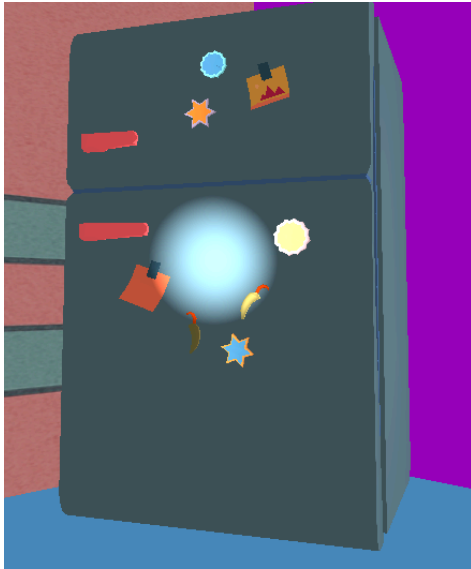
//Mentita
model = modelaux;
model = glm::translate(model, glm::vec3(-4.8f, -4.9f, 11.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
mentita.RenderModel();

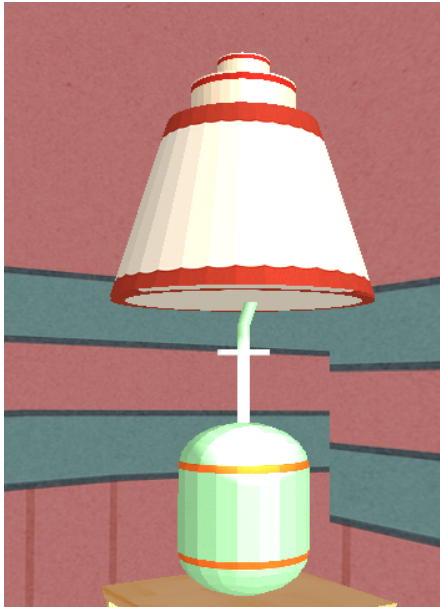
//Sillon cuarto 1
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -3.9f, 18.8f));
model = glm::scale(model, glm::vec3(65.0f, 70.0f, 65.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sillon2.RenderModel();

```

- Elementos del cuarto 2

Elementos





Importar modelos

```
/// Cuarto 2 ///
```

```
Model lampara;  
Model guitar;  
Model bmo;  
Model refri;  
Model radio;
```



```
// cuarto 2
lampara = Model();
lampara.LoadModel("Models/lampara.obj");

guitar = Model();
guitar.LoadModel("Models/guitar.obj");

bmo = Model();
bmo.LoadModel("Models/bmo.obj");

refri = Model();
refri.LoadModel("Models/refri.obj");

radio = Model();
radio.LoadModel("Models/radio.obj");
```

Implementación

```
//Mesa para lampara
model = modelaux;
model = glm::translate(model, glm::vec3(4.3f, -3.8f, -17.5f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.15f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
mesa.RenderModel();

//Lampara
model = modelaux;
model = glm::translate(model, glm::vec3(4.3f, -3.5f, -17.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
lampara.RenderModel();

//Guitarra
model = modelaux;
model = glm::translate(model, glm::vec3(5.0f, -3.95f, -10.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, 20 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
guitar.RenderModel();

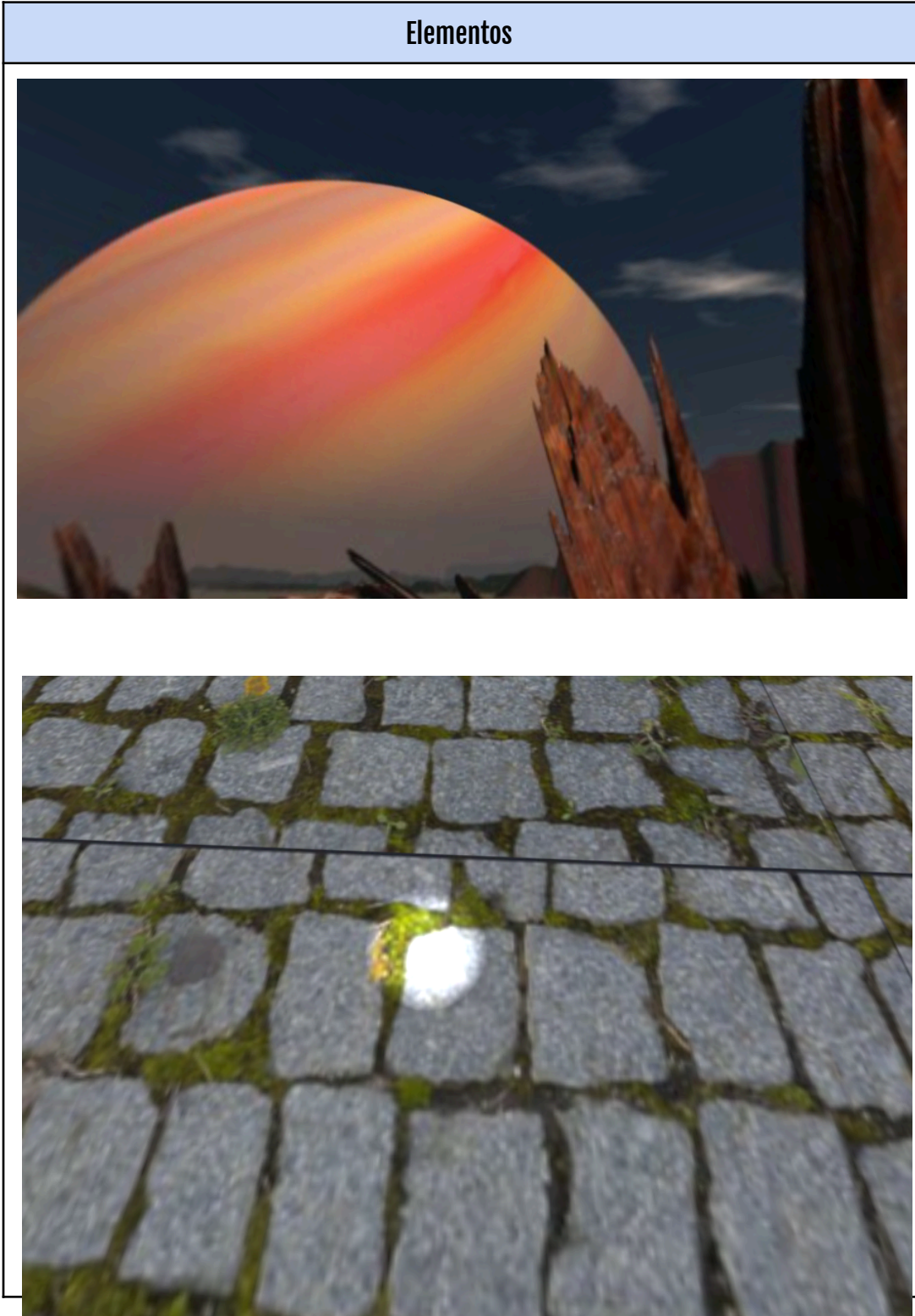
//Bmo
model = modelaux;
model = glm::translate(model, glm::vec3(-3.0f, -2.73f, -10.0f));
model = glm::rotate(model, 135 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
bmo.RenderModel();

//Refri
model = modelaux;
model = glm::translate(model, glm::vec3(4.7f, -4.95f, -2.5f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
refri.RenderModel();

//Radio
model = modelaux;
model = glm::translate(model, glm::vec3(1.2f, -14.75f, -17.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
radio.RenderModel();
```

--

- Escenario y skybox



Implementación

```
Texture plainTexture;  
Texture pisoTexture;  
Texture AgaveTexture;  
Texture FlechaTexture;
```

```
plainTexture = Texture("Textures/plain.png");  
plainTexture.LoadTextureA();  
pisoTexture = Texture("Textures/piso.tga");  
pisoTexture.LoadTextureA();
```

```
std::vector<std::string> skyboxFaces;  
skyboxFaces.push_back("Textures/Skybox/sp2_rt.png");  
skyboxFaces.push_back("Textures/Skybox/sp2_lf.png");  
skyboxFaces.push_back("Textures/Skybox/sp2_dn.png");  
skyboxFaces.push_back("Textures/Skybox/sp2_up.png");  
skyboxFaces.push_back("Textures/Skybox/sp2_bk.png");  
skyboxFaces.push_back("Textures/Skybox/sp2_ft.png");  
  
skybox = Skybox(skyboxFaces);
```

```
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));  
model = glm::scale(model, glm::vec3(30.0f, 1.0f, 30.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));  
pisoTexture.UseTexture();  
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);  
  
meshList[2]->RenderMesh();
```

- Cámara sintética

Implementación

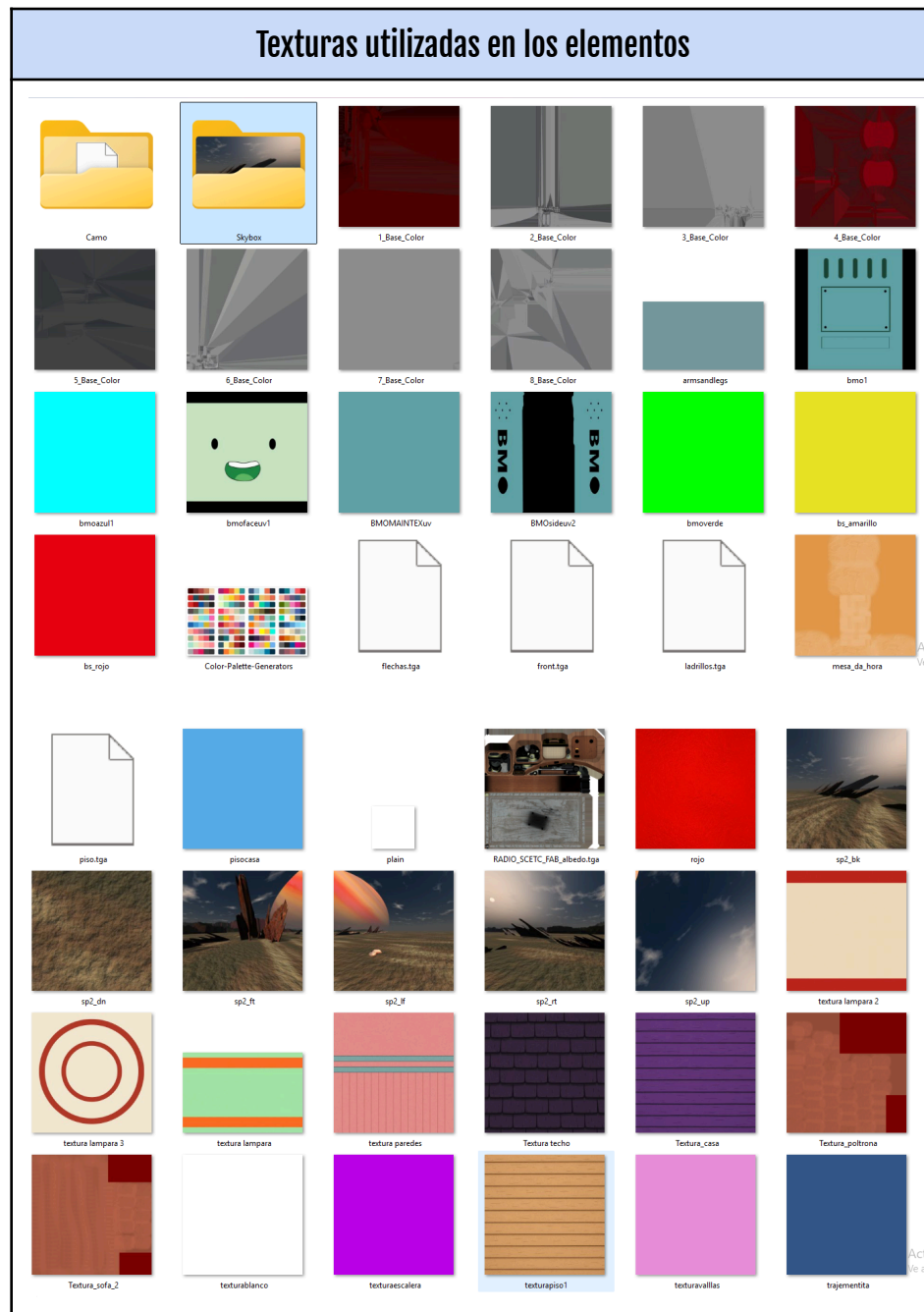
```
Camera camera;
```

```
camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.5f, 0.5f);
```

```
//Recibir eventos del usuario  
glfwPollEvents();  
camera.keyControl(mainWindow.getKeys(), deltaTime);  
camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());
```

- **Texturizado**

(Se puede observar en las imágenes mostradas en el documento)



- Iluminación

Luz direccional



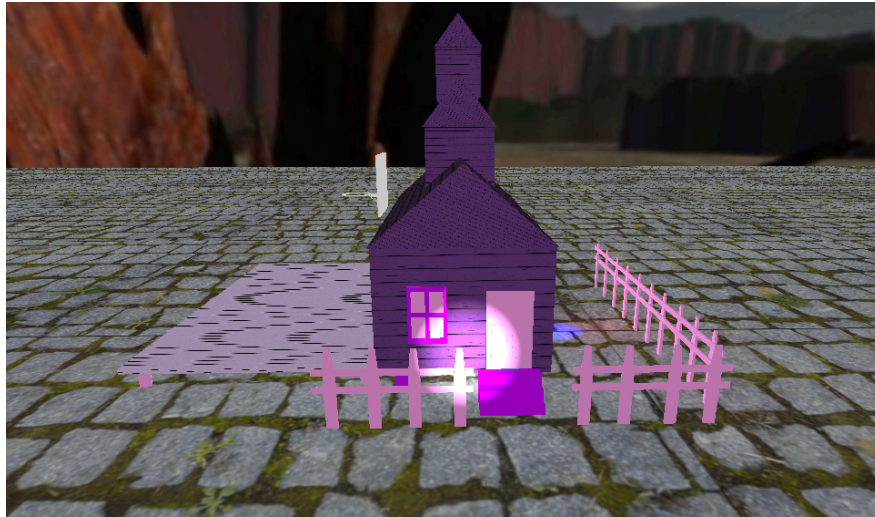
Implementación

```
// luz direccional
DirectionalLight mainLight;

//luz direccional, sólo 1 y siempre debe de existir
mainLight = DirectionalLight(1.0f, 1.0f, 1.0f,
    0.8f, 0.5f,
    0.0f, 0.0f, -1.0f);

//información al shader de fuentes de iluminación
shaderList[0].SetDirectionalLight(&mainLight);
shaderList[0].SetPointLights(pointLights, pointLightCount);
shaderList[0].SetSpotLights(spotLights, spotLightCount);
```

Luz tipo cono



Implementación

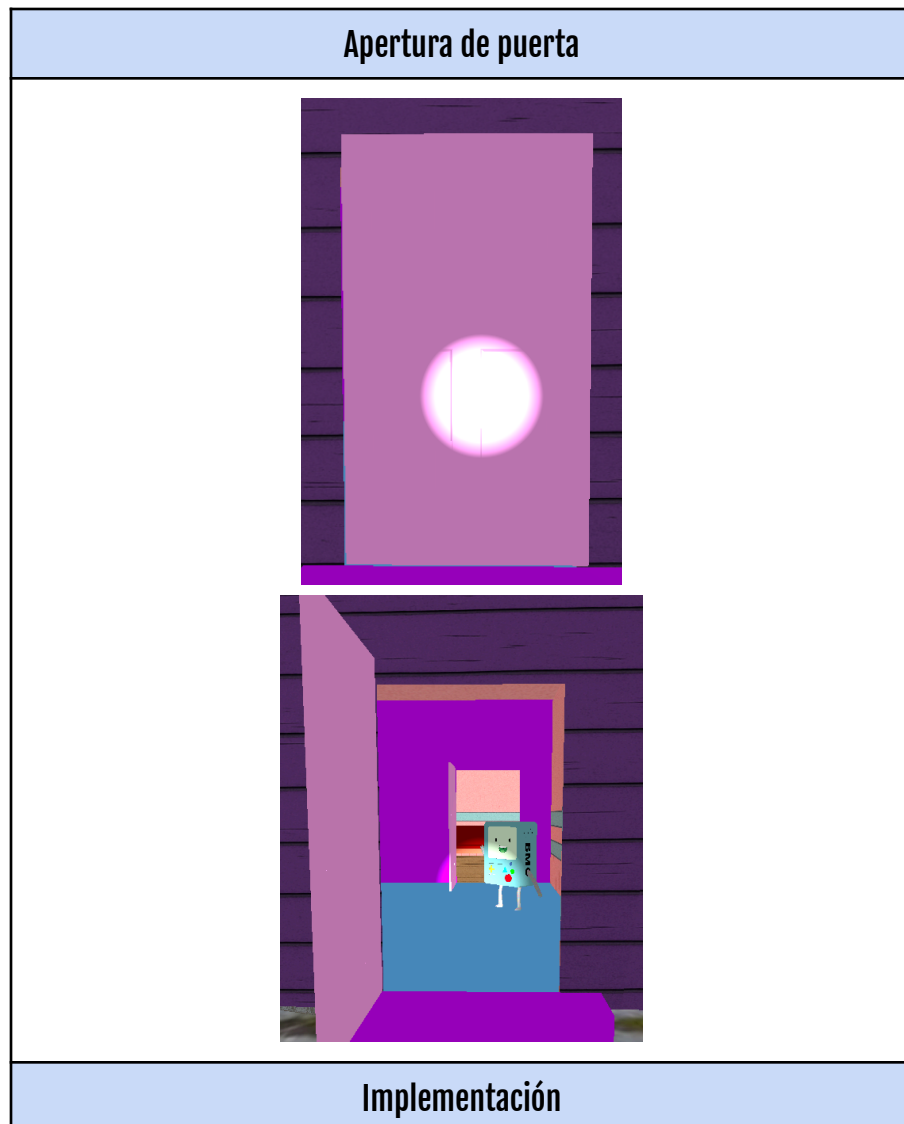
```
SpotLight spotLights[MAX_SPOT_LIGHTS];
```

```
unsigned int spotLightCount = 0;  
//linterna  
spotLights[0] = SpotLight(1.0f, 1.0f, 1.0f,  
    0.0f, 2.0f,  
    0.0f, 0.0f, 0.0f,  
    0.0f, -1.0f, 0.0f,  
    1.0f, 0.0f, 0.0f,  
    5.0f);  
spotLightCount++;
```

```
// luz ligada a la cámara de tipo flash  
lowerLight = camera.getCameraPosition();  
lowerLight.y -= 0.3f;  
spotLights[0].SetFlash(lowerLight, camera.getCameraDirection());
```

```
//información al shader de fuentes de iluminación  
shaderList[0].SetDirectionalLight(&mainLight);  
shaderList[0].SetPointLights(pointLights, pointLightCount);  
shaderList[0].SetSpotLights(spotLights, spotLightCount);
```

- Animación 1



```

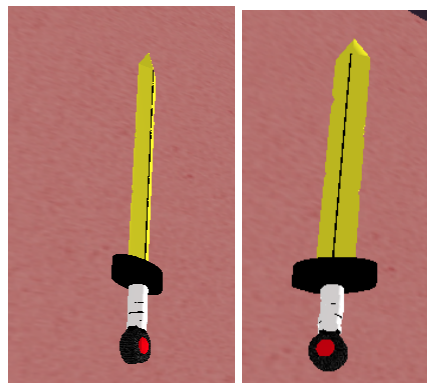
// Movimiento rotacion
if (mainWindow.getMover()) {
    if (rotpuerta > -90.0f) {
        {
            rotpuerta -= rotpuertaOffset * deltaTime;
        }
    }
    else {
        if (rotpuerta < 0.0f) {
            {
                rotpuerta += rotpuertaOffset * deltaTime;
            }
        }
    }
}

//Puerta 1
model = modelaux;
model = glm::translate(model, glm::vec3(-1.7f, -3.1f, -20.0f));
model = glm::rotate(model, rotpuerta * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta.RenderModel();

//Puerta 2
model = modelaux;
model = glm::translate(model, glm::vec3(-1.7f, -3.45f, 0.0f));
model = glm::rotate(model, rotpuerta * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
puerta.RenderModel();

```

Levitación de la espada



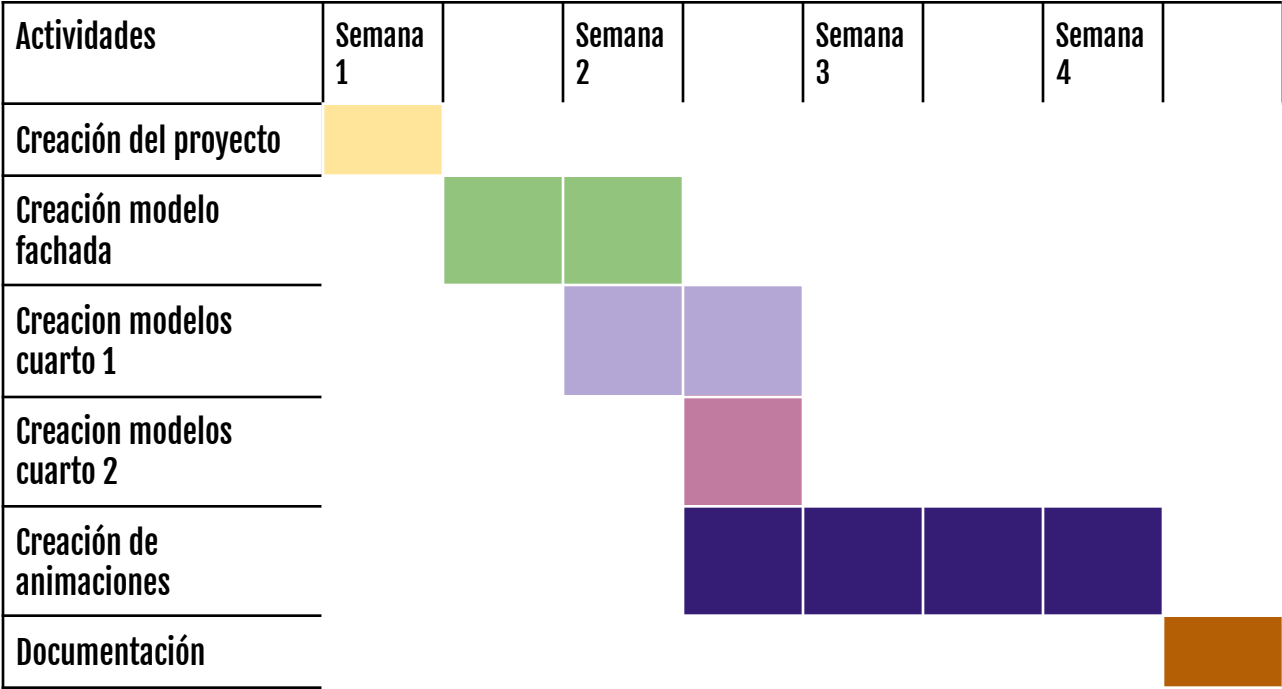
Implementación


```
///////// Cuarto 1 ///////////

anguloEspada += velocidad * deltaTime;
if (anguloEspada >= 360.0f) {
    anguloEspada -= 360.0f; // reinicia suave, sin saltos
}

//Espada
model = modelaux;
model = glm::translate(model, glm::vec3(6.8f, 0.5f, 5.5f));
model = glm::translate(model, glm::vec3(0.0f, 0.0 + 0.5 * sin(glm::radians(2 * angulovaria)), 6.0));
model = glm::rotate(model, anguloEspada * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
espada.RenderModel();
```

● Diagrama de Gantt.



Conclusiones

- 320109766

Tras haber terminado algunas de las actividades del proyecto, pude aplicar los conocimientos que adquirí en cada práctica de laboratorio. Este proyecto fue un gran reto, al final por diversas

complicaciones no logré realizar todas las animaciones de manera adecuada. Para desarrollarlo usé varias herramientas, como Blender para el modelado, Gimp para las texturas y Git junto con GitHub para llevar un buen control del proyecto.

En general, esta materia me pareció muy entretenida y, aunque la computación gráfica no es uno de mis fuertes, traté de completar las actividades correctamente.

Bibliografía

- Holubov, P. (2020, 1 octubre). *Vintage Retro Radio* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/vintage-retro-radio-26fb0de1015d4570bac28cd8b1fb0a62#download>
- Autor desconocido. (s. f.). *Marceline's AX Bass* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/marcelines-ax-bass-417178d709774642b0d5b5a02181caa2>
- Mendes, O. (s. f.). *Ice King* [Modelo 3D]. Sketchfab. <https://sketchfab.com/models/998ea9be02194e618ae3551d36764f1d>
- luisguilhermefoliveira. (2021, 7 de agosto). *Sword Finn* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/sword-finn-8ffcf2736553498ead2d8634ac6d574c>
- Autor desconocido. (s. f.). *Mesa da hora de madeira* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/mesa-da-hora-de-madeira-446665edffa24c5f806a8ab507682b85>
- Autor desconocido. (s. f.). *BMO* [Modelo 3D]. Sketchfab. <https://sketchfab.com/3d-models/bmo-0821dc59eedd469db946b8adb202af11>